```
import heapq

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
    'C': [('D', 1)],
    'D': []
}

h = {'A': 4, 'B': 3, 'C': 1, 'D': 0}

def astar(start, goal):
    queue = [(h[start], 0, start, [start])]
    best = {start: 0}

    while queue:
        f, g, node, path = heapq.heappop(queue)

        if g != best.get(node):
            continue

        print("Node:", node, "g =", g, "h =", h[node], "f =", f)

        if node == goal:
            print("Optimal Path:", " -> ".join(path))
            print("Minimum Cost:", g)
            return

        for neighbor, cost in graph[node]:
            new_g = g + cost

            if new_g < best.get(neighbor, float('inf')):
                best[neighbor] = new_g
                heapq.heappush(queue, (new_g + h[neighbor],
                                       new_g, neighbor, path + [neighbor]))

astar('A', 'D')
```

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:1
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for me
>>>
========== RESTART: C:/Users/user/Downloads/AIE
========
Node: A g = 0 h = 4 f = 4
Node: B g = 1 h = 3 f = 4
Node: C g = 3 h = 1 f = 4
Node: D g = 4 h = 0 f = 4
Optimal Path: A -> B -> C -> D
Minimum Cost: 4
>>>
```